Synchronization

Synchronization in C++

Stefano Quer
Dipartimento di Automatica e Informatica
Politecnico di Torino

License Information

This work is licensed under the license

CC BY-NC-ND 4.0

Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

BY: Credit must be given to you, the creator.

NC: Only noncommercial use of your work is permitted.
Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

ND: No derivatives or adaptations of your work are permitted.

To view a copy of the license, visit:
https://creativecommons.org/licenses/by-nc-nd/4.0/?ref=chooser-v1

Introduction

For condition variables, see Section u06s06

- C++11 introduced synchronization primitives
  - Mutexes and condition variables
    - A semaphore could be implemented using a mutex and a condition variables
- C++14 and C++17 extended the support
  - For example, introduced recursive_mutex, shared_mutex, shared_lock, etc.
- C++20 introduced semaphores
  - Binary semaphores in binary_semaphore
  - Counting semaphores in counting_semaphore

Mutexes in C++

For more operations see the reference documentation

- The complete library now defines several classes
  - For mutexes

Type	Meaning
std::mutex	Mutual exclusion. Protect a shared data from being accessed by multiple threads. The semantic is exclusive and non-recursive.
std::recursive_mutex	As the a mutex but offer recursive ownership semantics. A recursive function can acquire the mutex more than once.
std::shared_mutex	As a mutex but allows a shared or an exclusive locks.
std::timed_mutex	As a mutex but with the possibility to use timeout during locking.

Mutexes in C++

For more operations see the reference documentation

- Member functions are common (mutexes & locks)
  - Some of them (e.g., timed locks) are not always available

Type	Meaning
lock	Locks (i.e., takes ownership of) the associated mutex.
try_lock	Tries to lock (i.e., takes ownership of) the associated mutex without blocking
try_lock_for	Attempts to lock (i.e., takes ownership of) the associated mutex, returns if the mutex has been unavailable for the specified time duration.
try_lock_until	Tries to lock (i.e., takes ownership of) the associated mutex, returns if the mutex has been unavailable until specified time point has been reached.
unlock	Unlocks (i.e., releases ownership of) the associated mutex.

Mutex

The same thread must lock and unlock the mutex

- Mutex offers exclusive, non-recursive ownership semantics
- They implemented in the class std::mutex
  - A calling thread owns a mutex from the time that it successfully calls either lock or try_lock until it calls unlock
  - When a thread owns a mutex, all other threads will block (for calls to lock) or receive a false return value (for try_lock) if they attempt to claim the mutex
  - A calling thread must not own the mutex prior to calling lock or try_lock

Mutex

- The behavior of a program is undefined if
  - A mutex is destroyed while still owned by any threads
  - A thread terminates while owning a mutex
- The class std::mutex is **neither** copyable nor movable

Example

Revisitation of the example in Section u05s04

```
std::mutex m;

void safe_print (int i) {
  m.lock ();
  std::cout << i;
  m.unlock ();
}
```

```
std::vector<std::thread> threadPool;

for (int i = 1; i <= 9; ++i) {
  threadPool.emplace_back([i] { safe_print(i); });
}

for (auto& t : threadPool) {
  t.join();
}
```

Digits 1 to 9 are printed (unordered)

Recursive Mutex

- A recursive mutex is a regular mutex that additionally allows a thread that holds the mutex to lock it again
  - Useful for **recursive** functions or functions that call each other and use the same mutex
- Recursive mutexes are implemented in the class std::recursive_mutex
  - The unlock function must still be called once for each lock

Example

Function f1 calls f2 (but f2 can be called directly)

```
std::recursive_mutex rm;

void f1() {
  rm.lock();
  std::cout << "foo\n";
  f2();
  rm.unlock();
}


void f2() {
  rm.lock();
  std::cout << "bar\n";
  rm.unlock();
}
```

Function f1 must protect IO and then calls function f2

Function f2 must also lock and unlock IO because can be called directly (not through f1)

A standard mutex will block f2 forever

Shared Mutex

- Introduced in C++17
  - Are implemented in the class std::shared_mutex
- A shared mutex is a mutex that allows shared and exclusive locks
  - An **exclusive** lock can be obtained by a single thread
    - Are granted by the member functions **lock** and **unlock**
  - A **shared** can be obtained by more than one thread
    - Shared locks are granted by the member functions **lock_shared** and **unlock_shared**

Shared Mutex

- It is also possible to use the "try" versions during locking and returns true or false
  - Functions **try_lock** try to get an exclusive lock
  - Function **try_lock_shared** try to get a shared lock
- Shared mutexes are mostly used to implement reader and writer locks
  - Readers use shared locks
  - Writers use exclusive locks

In POSIX these mutexes are called Reader-Writer Locks!

Example

Reader and Writer Protocol

```
int value = 0;
std::shared_mutex sm;

std::vector<std::thread> tv;

for (int i = 0; i < 5; ++i)
  tv.emplace_back([&] {
    sm.lock_shared();
    ... use value ...
    sm.unlock_shared();
  })

for (int i = 0; i < 5; ++i)
  tv.emplace_back([&] {
    sm.lock();
    ++value;
    sm.unlock();
  })
```

Run **reader threads** that can **share** the critical section

Run **writer threads** that must acquire an **exclusive** lock on the critical section

Timed mutex

- The library std::timed_mutex, similarly to std::mutex, offers exclusive, non-recursive ownership semantics
- In addition, std::timed_mutex provides the ability to attempt to claim ownership of a std::timed_mutex with a timeout via the member functions
  - try_lock_for
  - try_lock_until

RAII wrappers

- Mutexes can be thought of resources that must be acquired and freed with lock and unlock
- Thus, they are easily subject to errors
  - Errors can be difficult to trace back

T_i

```
m.lock();
...
m.unlock();
```

T_j is buggy and make problems to the entire application, possibly locking also T_i for ever

T_j

```
m.unlock();
...
m.unlock();
```

T_j

```
m.lock();
...
m.lock();
```

T_j

```
m.unlock();
...
m.lock();
```

RAII wrappers

For more operations see the reference documentation

- To avoid these problems, it is possible to use RAII paradigm, i.e., wrappers for mutexes

Type	Meaning
std::lock_guard	A mutex wrapper that provides a convenient RAII-style mechanism for owning a mutex for the duration of a scoped block.
std::unique_lock	A RAII wrapper for std::mutex. A mutex wrapper allowing deferred locking, time-constrained attempts at locking, recursive locking, transfer of lock ownership, and use with condition variables.
std::shared_lock	RAII wrapper for std::shared_mutex. A shared mutex wrapper allowing deferred locking, timed locking and transfer of lock ownership.
std::scoped_lock	RAII wrapper for owning zero or more mutexes (contestually) with no deadlock.

RAII wrappers

- They call lock in the constructor and unlock in the destructor
  - They are movable to "transfer ownership" of the locked mutex
  - They also have the member functions **lock** and **unlock** to manually control the mutex

Example

Lock guards

- When
  - A lock guard object is created, it attempts to take ownership of the mutex it is given
  - Control leaves the scope, the lock guard is destructed and the mutex is released

```
#include <iostram:
#include <thread>
#include <mutex>

std::mutex m;

void t () {
  const std::lock_guard<std::mutex> lock(m);
  ...
  return;
}
```

Creates a lock guard and acquires m

The mutex m is automatically released when the lock gets out of scope

Example

Unique locks with mutex

```
#include <iostram:
#include <thread>
#include <mutex>

std::mutex m;

void f1 () {
    std::unique_lock l{m};
    ...
}
```

Create a lock l on the mutex m
m.lock() is called for immediate locking

m.unlock() is called

Alternative definition

```
std::unique_lock<std::mutex> l{m};
```

It also guarantees unlocking in case an exception is thrown

Example

Unique locks with shared mutex

```
#include <iostram:
#include <thread>
#include <mutex>

std::shared_mutex sm;

void f1 () {
    std::unique_lock l{sm};
    ...
}
```

Create a lock l on the shared mutex sm
sm.lock() is called for immediate locking

sm.unlock() is called

Alternative definition

```
std::unique_lock<std::shared_mutex> l{sm};
```

It also guarantees unlocking in case an exception is thrown

Example

Unique locks with deferred locking

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex m1;
std::mutex m2;

void update (int num) {
  std::unique_lock lock1{m1, std::defer_lock};
  std::unique_lock lock2{m2, std::defer_lock};

  std::lock(lock1);
  std::lock(lock2);

  num += (int) rand();

  return;
}
```

Don't actually take the locks on m1 and m2

std::lock(lock1,lock2);
lock both unique locks without deadlock
(due to the **order**)

m1 ad m2 are **automatically** unlocked

Scoped locks
Part A

Example

- When threads use multiple mutexes
  - It is easy to cause a deadlock if different threads try to lock the mutexes in a different order
  - The solution is to force a consistent order

threadA can get the locks for m1 and m2
threadB gets a lock on m3 …
deadlock

```
std::mutex m1, m2, m3;

void threadA() {
  std::unique_lock l1{m1}, l2{m2}, l3{m3};
}
void threadB() {
  std::unique_lock l3{m3}, l2{m2}, l1{m1};
}
```

The order is not consistent with threadB

Scoped locks
Part B

Example

- When threads use multiple mutexes
  - Sometimes, it is not possible to guarantee a consistent order
  - The class **std::scoped_lock** takes any number of mutexes and locks them using a deadlock-free algorithm

```
std::mutex m1, m2, m3;

void threadA() {
  std::scoped_lock l{m1, m2, m3};
}
void threadB() {
  std::scoped_lock l{m3, m2, m1};
}
```

It is generally very time-inefficient.
Use with care.

Semaphores in C++

For more operations see the reference documentation

- Semaphores in C++ have been introduced with C++20
  - They contain a counter initialized by the constructor
  - When the counter is zero, **acquire** blocks until the counter is incremented

Type	Meaning
counting_semaphores	Synchronization primitive that can control access to a shared resource. Unlike a mutex a counting semaphore allows more than one concurrent access to the same resource.
binary_semaphores	A specialization form of the counting semaphore with the maximum value being 1. May be more efficiently than the default counting semaphore.

Semaphores in C++

For more operations see the reference documentation

- Member functions are common (mutexes & locks)
  - Unlike mutex semaphores **are not tied to threads**, i.e., acquiring a semaphore can occur on a different thread than releasing the semaphore
  - All operations on semaphore can be performed concurrently and without any relation to specific threads of execution

Type	Meaning
release	Increments the internal counter and unblocks acquirers.
acquire	Decrements the internal counter or blocks until it can.
try_acquire	Tries to decrement the internal counter without blocking.
try_acquire_for	Tries to decrement the internal counter, blocking for up to a duration time.
try_acquire_until	Tries to decrement the internal counter, blocking until a point in time.

Example

```
#include <chrono>
#include <iostream>
#include <semaphore>
#include <thread>

std::binary_semaphore m2t{0}, t2m{0};

void tf() {
  m2t.acquire();
  std::cout << "[thread] Got signal\n";
  std::this_thread::sleep_for(3s);
  std::cout << "[thread] Send signal\n";
  t2m.release();
}

int main() {
  std::thread thr(tf);
  std::cout << "[main] Send signal\n";
  m2t.release();
  t2m.acquire();
  std::cout << "[main] Got signal\n";
  thrWorker.join();
}
```

Conters are set to 0 (non-signaled state)

m2t = main to thread
t2m = thread to main

Conclusions

- As in all other standards, synchronization must be used with care
  - For each call to **lock** or **acquire**, **unlock** or **release** must be called exactly once
  - For mutex function **lock** and **unlock** must be called by the **same** thread
  - Mutexes and semaphores are usually neither copyable nor movable
    - It is not trivial to create dynamic data structure of them but using dynamic memory allocation
  - Pay attention to deadlock
    - Prefere RAII versions when possible